Simulating collision of two elastic disks in 2-D using Material Point Method

Alwin Anto, aa93428, TACC ID: alwinanto, email : alwinanto@utexas.edu

April 2025

Github Repo

1 Introduction

The objective of this project is to verify the conservation of energy of a system of two elastic disks colliding in 2-D. The problem has been solved using Material Point Method. As a result we observe the total energy of the system to remain constant throughout while the energy transfer from kinetic to strain energy during the collision. The code is written in C++ and made parallel using MPI. An additional goal of this project is to conduct a weak scaling study by increasing the number of processes proportionate to the number of particles used in the simulation

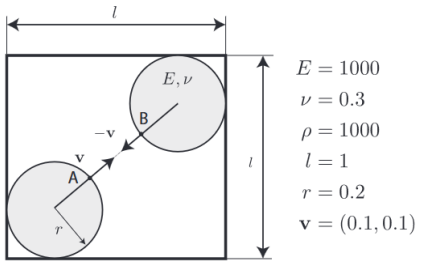


Figure 1: Elastic disks

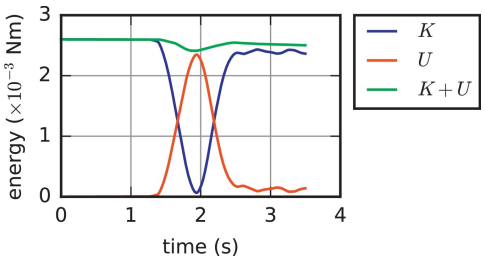


Figure 2: Energy vs time

2 Material Point Method

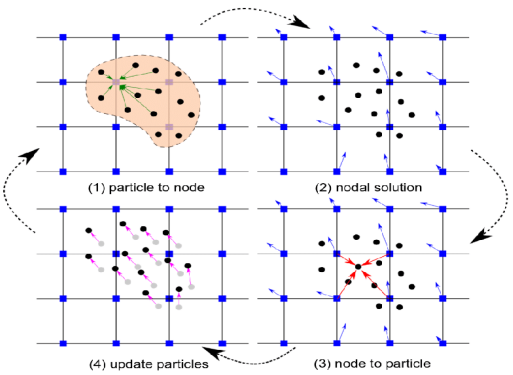


Figure 3: Material Point Method

This mesh-based method solves the governing equations on fixed background grid points, while material points (particles) move independently through the domain—i.e., Lagrangian particles on an Eulerian mesh. It is often compared to the finite element method (FEM). Although typically less accurate than FEM, it offers faster computation and handles contact problems more robustly due to which the method is gaining popularity. The key steps in the method are outlined below.

1. **Initialize grid and material points (particles):**
   Set up the computational grid and initialize particle positions, velocities, masses, volumes, deformation gradients, and material parameters.

2. **Transfer particle data to grid nodes:**
   Use shape functions to map mass, momentum, and forces from particles to nearby grid nodes.

3. **Update nodal velocities and forces:**
   Use *forward Euler scheme* to update nodal velocities from the forces present. Enforce boundary conditions.

4. **Transfer updated grid information back to particles:**
   Update particle velocities and momentum from nodal data using the same weights used in Step 2

5. **Update particle positions,deformation gradient and compute stress:**
   Use the new velocities to update position, deformation gradients, and compute stress via constitutive relations. Calculate energy and other quantities of interest

6. **Reset the grid and repeat steps 2–5:**
   Reset grid values and continue the loop for the duration of the simulation.

3 MPI vs OpenMP

For small problems we can utilize the embarrassingly parallel nature of the problem by using OpenMP to parallelize the code that is written for serial implementation. However this restricts us to a certain number of particles as memory would be a threshold and a method like MPM is preferred in most cases when we make use of millions of particles. A hybrid method of OpenMP within nodes and MPI across nodes or shared memory MPI (using MPI WIN ALLOCATE SHARED) for processes within the same node was looked at but avoided as handling race conditions would slow down the run rime significantly. Especially at the time of collision all the material points will be concentrated over fewer grids and would be trying to update the same gridpoints. Hence the decision was made to proceed with an implementation in MPI.

4 Parallelism by partitioning

Two aspects arise here - distributing the material points and grid points over all processes.

1. **Material points/particles:** Generally in MPM as well in our application the number of particles is several orders higher than the grid. The size of a single object of the particle class would also be high due to all the quantities we need to track for the particle. Equally dividing them is hence one of the key problems to address. The circular domain of particles is split into equal angles for each of the process. This ensures equal distribution but when number of processes are high this tends to be more like 1-D partitioning rather than 2-D (square domain partitions) which would be better due to data locality and lesser communication requirement

```
// equal pieces of the pie!
double start_angle = 2 * M_PI * procno / nprocs;
double end_angle = 2 * M_PI * (procno + 1) / nprocs;

for ( fx = -fRadius ; fx < fRadius; fx+=gap)
  for (fy = -fRadius ; fy < fRadius; fy+=gap) {
     if (fx * fx + fy * fy < fRadius * fRadius) {
        double angle = std::atan2(fy, fx);
        if (angle < 0) angle += 2 * M_PI; // -pi to pi -> 0 to 2
            pi
        if (angle >= start_angle && angle < end_angle)
          create one particle ; }}
```

Listing 1: Assign material points based on angle partition

2. **Grid points:** A naive equi-partioning of grid points doesn't lead to parallelism. Since grid stays constant and particles move significantly throughout the simulation most of the grid points aren't involved in the computation at a given time which defeats the purpose spreading the grid over various cores. Hence a dynamic partitioning is required but this is harder. Further if this partitioning does not match the particle partitioning then the amount of communication required and figuring which process owns which particle adds to the difficulty (since particles move). The necessity of 2-D square partitioning of particles is further highlighted here as figuring out the communication is easier. However the solution for the 2-D partitioning I had would have lead to disproportionate splitting of the material particles and that is worse as number of particles are much higher and have more computations related to it (Grid points only need to update velocity and position). For our problem the grid size varies from 30x30 to 70x70 whereas number of material points vary from 10^4 to 10^7. Hence each process has a grid copy and ALLREDUCE makes sure the grid copy each process has combines information of all required particles from all the processes

```
// information[] contains mass, velocity and force that are
    needed to solve the governing equation
  MPI_Allreduce(MPI_IN_PLACE,information.data(),5*thisGrid.iNodes
     , MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

// Get the energy from all processes at the end by root for
    tracking energy over time
    MPI_Reduce(Energy.data(), GlobalEnergy.data(),2*(time_steps),
        MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Listing 2: Combining grid information and other collective communications used

5 Results

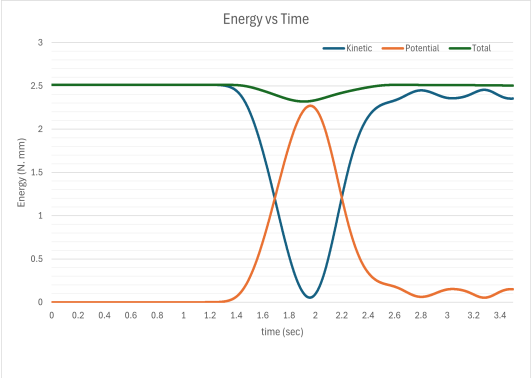


Figure 4: Evolution of energy over time

The results we obtained match with the literature available (1). We can see the total energy remains constant almost throughout the simulation except when the balls collide where some error is observed. We can also see how post collision the vibration of the disks have been captured. And hence we see the oscillation of the strain and kinetic energies after collision. In reality these vibrations get damped out due to friction from air and ground leading to energy dying down.

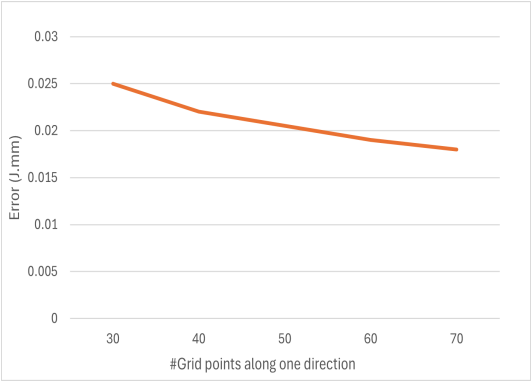


Figure 5: Mean error vs number of grid points in one direction ($\sqrt{N_{gps}}$)

Here in fig 5 can see that increasing the error is decreasing as we increase the number of grid points. This is expected and good but the rate of decrease is very slow and this helps with our argument that increasing gridpoints doesn't take priority

Time (sec)	112	110	111	111	112
$\text{Nx}_{gridpoints}$	30	40	50	60	70

Table 1: Time and number of gridpoints ($6.4 \times 10e^5$ material points, 64 cores)

Note that N gridpoints along x direction implies N^2 total gridpoints. In table 1 we can see that increasing number of gridpoints does not increase the time. This is due to the way we have implemented the algorithm. We are using all reduce to combine grid data and here for us the main component is the latency. This is because the size of the data we are sending is not that big. Note that once grids are made too fine we won't be able to run the simulation due to memory constraints but for our use-case we don't require taht level of fineness.

6 Scaling Study

Due to the memory bound nature of our problem doing a weak scaling study makes more sense for us. Further we have been able to divide the material points equally among the cores which is crucial for the accuracy of this study. Now since gridpoints are kept constant we need to increase materials points while increasing the number of cores for carrying out the weak scaling study. The input argument we use to decide number of material points is "o" (offset i.e distance between two material points). Each material point occupies an area of o^2 and total area of circle is πr^2. Hence we have total material points

$$N_{total}^{mp} = \frac{\pi r^2}{o^2}$$

$$\implies \frac{N_{total}^{mp}}{n_{cores}} = \frac{\pi r^2}{n_{cores} o^2} = const$$

$$\implies o = \frac{const}{\sqrt{n_{cores}}}$$

Cores	Material points per core	Total Material Points	Time (sec)
1	10000	10^4	106
2	10002	2×10^4	108
4	10018	4×10^4	107
16	10028	1.6×10^5	109
32	9990	3.2×10^5	109
64	10056	6.4×10^5	113
128	9972	1.27×10^6	115
256	10070	2.56×10^6	115
512	9990	5.12×10^6	124

Table 2: Performance data: material points scale with cores and grid points

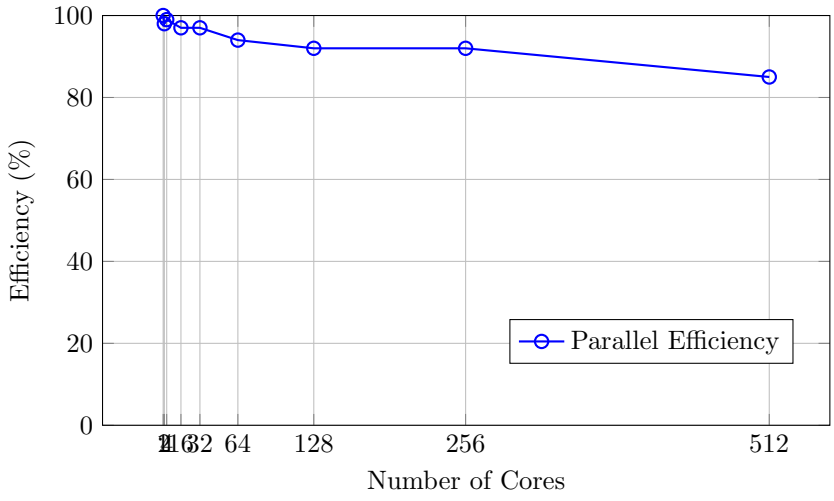


Figure 1: Parallel Efficiency vs. Number of Cores

The constant we have taken is 0.005. The weak scaling graph is more or less within our expectations. 100% efficiency or more is not usually expected unless there are cache hits that improve in parallel due to data locality etc. In usual cases more the cores and nodes involved the efficiency starts dropping due to difiiculty in memory access

7 Conclusion

Regards to the problem at hand we were able to successfully simulate the elastic collision, transfer and conservation of energy of a 2 body system in 2-D.The parallelization strategy focused on distributing the computationally intensive material points across processes, while maintaining a full copy of the relatively smaller grid on each process. This helped us to avoid complex and possibly costly dynamic partitioning of grid points. Our weak scaling study showed that the implementation is efficient and scalable, especially considering the memory-bound nature of MPM.

Some of the limitations and areas that look exciting to me include changing the angular partitioning of particles to something else that can exploit data locality better. Second, a hybrid MPI+OpenMP model needs to be compared with to see by using atomics and avoiding race conditions am I actually able to imorove efficiency. Third, although grid resolution was not the main bottleneck, it still influences accuracy, and adaptive mesh refinement would be needed for harder problems. Better material points partitioning would also bring me closer to this goal

Overall, the implementation demonstrates that MPM is not only a viable but also an efficient method for simulating large-scale, dynamic contact problems in parallel environments.

References

[1] Sinaie, Sina Nguyen, Vinh Phu Thanh Nguyen, Chi Bordas, Stéphane. (2017). *Programming the material point method in Julia. Advances in Engineering Software*

[2] Victor Eijkhout. *The Art of HPC*

Appendix

1. **Timing**: Made sure to exclude the writing to file part from timing

```
    MPI_Init(&argc, &argv);
    double start_time = MPI_Wtime();
    .
    .
    double end_time = MPI_Wtime();
    Writing to file for further analysis
```

2. **Use of derived data type:** Useful to send all information(mass,force,velocity) simultaneously.

```
        MPI_Datatype MPI_Point;
        MPI_Type_contiguous(5, MPI_DOUBLE, &MPI_Point);
        MPI_Type_commit(&MPI_Point);
        .
        .
        MPI_Type_free(&MPI_Point);
```

3. **Relevant equations governing the physics and other matters** beginitemize Efficiency $E = \frac{T_{ncores}}{T_1}$

- **Position and velocity update**

$$\mathbf{v}_p^{n+1} = \mathbf{v}_p^n + \sum_i N_{ip} \frac{\mathbf{f}_i}{m_i} \Delta t$$

$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \sum_i N_{ip} \mathbf{v}_i \Delta t$$

- **Deformation gradient and strain increment**

$$\dot{\mathbf{F}}_p = \sum_i \mathbf{v}_i \otimes \nabla N_{ip}, \quad \mathbf{F}_p^{n+1} = \dot{\mathbf{F}}_p \cdot \mathbf{F}_p^n$$

$$\Delta \boldsymbol{\varepsilon}_p = \begin{bmatrix} F_{11} - 1 \\ F_{22} - 1 \\ F_{12} + F_{21} \end{bmatrix}$$

- **Stress strain relation (linear model)**

$$\Delta \boldsymbol{\sigma}_p = C \, \Delta \boldsymbol{\varepsilon}_p$$

$$C = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix}$$

- **Volume change**

$$V_p^{n+1} = \det(\mathbf{F}_p^{n+1}) \cdot V_p^0$$

- **Strain energy:**

$$U = \sum_p \frac{1}{2} \varepsilon_p^\top \sigma_p \, V_p$$

- **Kinetic energy:**

$$K = \sum_p \frac{1}{2} m_p \|\mathbf{v}_p\|^2$$